

Where an LLM Would Help — Priority Order

Ranked by overall project value (real gap closed, named pillar / judging criterion served, demo weight, private-set robustness, risk), **not** by public-score movement — which is ~zero for most of these. Judging is Technical Execution 35% / Innovation 20% / Impact 20% / Feasibility 15% / Presentation 10%; the automated TechnicalScore is <=35% of one criterion.

Design constraint throughout: the offline lexical core stays the guaranteed path and the score floor. Every LLM use is an **opt-in layer with a deterministic fallback**, so a network-restricted scoring run still gets ~0.92.

Tier 1 — do these

1. Clarification-question generation

Replace the fixed QUESTION_TEXT table + the repeated "Is there anything else that matters for this one?" with a natural, pool-aware question: "I'm seeing mostly leather and canvas belts here - do you have a preference?"

- ask_attribute **stays** "other". The English becomes *guidance*, not a

gate: the simulator ignores it and still returns the next 2 constraints of any type, so the score-optimal extraction is untouched. A real user answers the targeted question, or volunteers something more important - both fine.

- **Why #1**: the repetition is the most visibly bad thing about the agent, and

Pillar II asks for "structured, proactive clarification prompts that guide user convergence." Touches Presentation, Innovation and Impact - half of judging - at zero score risk.

Does this need an LLM? Mostly no - see section below. A template system over the facet distribution (machinery already in InfoGainPolicy._distributions) covers ~70-80% of cases. The LLM earns the last 20-30%: fluency on messy 3+ value splits, phrasing variety across a 10-turn session, stitching the disclosed constraints into the question, and the long tail (brands, odd category names).

Steer pool-based, not profile-based. "I saw you like fit" when the target is a necklace is a visible inconsistency in a demo (the profile is only 26.5% consistent with the target). "I'm seeing mostly X and Y" is grounded in what the agent actually retrieved and cannot be wrong the same way.

2. Recommendation explanations

"These all have a stainless-steel band and a date window" from the reranker's matched spans (state.query_spans() intersected with each shown product's text). A listed Innovation Direction ("transparent recommendation explanations"); cheap; high demo value; same per-turn model call as #1. Template works; the LLM only makes it read naturally.

Tier 2 — high value, more engineering

3. LLM semantic reranking (opt-in layer over the top ~15-20)

A cross-encoder (bge-reranker-base, ~1 GB, CPU-viable) or a listwise LLM prompt scores the final shortlist; **fused** with the existing rerank() score (not replacing it), local scorer as fallback.

- **Why**: Pillar I literally names "Multi-Route Retrieval -> LLM Semantic

Ranking." Having none is a Technical-Execution checkbox judges look for. Built as a fused opt-in layer it never regresses the offline 0.92, attacks the homogeneous_cluster / degenerate_card buckets where span coverage saturates, and is the one rerank change with a real ceiling on a *paraphrased* private set.

- **Risk**: medium - nondeterminism, latency, a semantic model can demote an

exact-string-match target. Mitigate: fusion with low weight, plateau-swept, disqualify on any public hit@10 regression.

- Note: a semantic-rerank-experiment branch already exists.

4. Paraphrase-robust constraint extraction

An LLM parses each customer message into structured slots, robust to synonyms / paraphrase / typos the regex + cue path misses ("something warm for winter" -> the current span matcher gets nothing).

- **Why:** the only LLM use that could move the *final* (private) score. The

private simulator "may paraphrase"; if it does, query_spans () collapses and the whole stack starves. Serves Pillar II and Feasibility.

- **Risk:** medium-high - a hallucinated constraint is worse than a missed one.

Guard: accept only extractions grounded in the literal message text; keep the regex path as a union, not a replacement.

Tier 3 - don't reach for an LLM here

use	why not
Dual-track routing	buying-vs-browsing classification is trivial without an LLM; the real gap is wiring the route into retrieval / timing / questioning - deterministic logic.
Adaptive orchestration	build stagnation detection (shortlist unchanged + no new disclosure -> switch strategy) as rules; an LLM orchestrator adds nondeterminism for a decision rules make better.
Personalized context distillation	the profile carries no signal (4 failed attempts); an LLM distilling noise gives distilled noise.
Query rewriting for retrieval	BM25 recall is already ~100% on the public set.

Does Tier 1 need an LLM? A closer look

The catalog is fixed and the facet vocabularies are small (src/facets.py: material ~25 values, colour ~20, style ~20, use_case ~18, size ~11). So the space of "which facet is the pool split on, and what are the top 2-3 values" is enumerable, exactly like the rest of the pipeline.

A no-LLM template system, concretely:

1. Reuse InfoGainPolicy._distributions(candidates) - it already returns the score-weighted value counts per attribute over the live pool.
2. Pick the attribute with the highest gain ratio that is not dead / asked
(this is InfoGainPolicy.scores minus the broad-question special-casing).
3. Take its top 2-3 values by mass.
4. Fill a phrase bank:
 - 2 values: "I'm seeing mostly {a} and {b} {noun} - any preference?"
 - 3 values: "There's a mix of {a}, {b} and {c} here - does one matter more?"
 - fallback: the current broad question.
5. Rotate 3-4 templates per shape so it doesn't sound identical every turn.

That covers the ~5 core facets cleanly and is deterministic, instant, free, and needs no infra. It is the honest first build and probably enough for the demo and the Presentation score.

What the LLM adds on top (the last 20-30%):

- fluency when the split is 3+ values or spans two attributes ("a mix of long-sleeve crew necks and short-sleeve v-necks");
- stitching in what the customer already said ("you mentioned leather - I'm now seeing both dress and casual belts, which?");
- phrasing variety that does not degrade over 10 turns;
- graceful handling of the long tail - brands, multi-word category leaves, values outside the facet vocab;
- the write-up line "we use a local instruct model to generate context-aware clarifications", which is itself an Innovation checkbox.

Recommendation: build the template system first (1-2 hours, no dependency), measure that it does not move the local score, ship it. Add the LLM as the opt-in polish layer with the template as fallback - so the guaranteed path is still deterministic and the demo degrades gracefully offline.

Practical build

One bge-reranker-base (cross-encoder) for #3, and one ~3-7B instruct model doing #1 + #2 + #4 in a single per-turn call (parse message -> generate question + explanation). Both offline, both with deterministic fallbacks. Dev on Codespaces (CPU) for everything except the reranker sweep; a Kaggle / RunPod GPU burst only if a local generative LLM is actually pursued.